LECTURE 2

Application Security Testing (AST)

Application Security Testing (AST) is the process of making applications resilient against cyber threats by identifying, analyzing, and fixing security vulnerabilities. This lecture covers the two primary strategies used in modern DevSecOps pipelines:

> SAST
> **Static Application Security Testing (SAST):** An inside-out, white-box testing approach that analyzes source code at rest without executing it.

> DAST
> **Dynamic Application Security Testing (DAST):** An outside-in, black-box testing approach that evaluates a running application by simulating real-world attacks against its endpoints.

IN THIS LECTURE

2.1 Static Application Security Testing (SAST)
2.2 Dynamic Application Security Testing (DAST)

2.1 Static Application Security Testing (SAST)

SAST tools integrate early in the Software Development Lifecycle (SDLC). They scan source files, build scripts, and configurations to catch structural flaws (e.g., weak crypto, hardcoded secrets, unprotected logic patterns) before the application is compiled or deployed.

In this lecture, we use **Bandit**, an open-source tool that parses Python code into an Abstract Syntax Tree (AST) to identify security bugs.

We will write a script containing three critical vulnerabilities:

1. Hardcoded Secret (CWE-259)
2. SQL Injection Vector (CWE-89)
3. Command Injection Vector (CWE-78)

2.1.1 Creating the Vulnerable Script

COLAB CELL 1 **Create Insecure File**

```
%%writefile vulnerable_app.py
import os
import sqlite3

# 1. HARDCODED SECRET
DB_PASSWORD = "SuperSecretPassword123!"

def unsafe_login(username, password):
    # 2. SQL INJECTION VULNERABILITY (String formatting)
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE username = '{username}' AND password =
'{password}'"
    cursor.execute(query)
    return cursor.fetchone()

def ping_host(user_ip):
    # 3. COMMAND INJECTION VULNERABILITY (os.system executing raw shell
commands)
    print(f"Pinging server...")
    os.system(f"ping -c 1 {user_ip}")
```

EXPECTED OUTPUT – CELL 1

```
Writing vulnerable_app.py
```

2.1.2 Running the SAST Scan

Next, we install bandit and execute it against our uncompiled text file.

COLAB CELL 2 **Run Bandit Scan**

```
# Install the security scanner tool
!pip install bandit

# Run static analysis against the vulnerable file
!bandit vulnerable_app.py
```

EXPECTED OUTPUT – CELL 2

```
Successfully installed bandit-1.8.2
[u'bandit'] [INFO] scanning paths: ['vulnerable_app.py']

Run started: 2026-10-09 14:44:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password:
'SuperSecretPassword123!'
   Severity: Medium   Confidence: High
   Location: vulnerable_app.py:5:14

>> Issue: [B608:sql_injection_string_vulnerability] Possible SQL injection
vector through string-based formatting.
   Severity: Medium   Confidence: High
   Location: vulnerable_app.py:11:12

>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell,
possible injection detected.
   Severity: High   Confidence: High
   Location: vulnerable_app.py:18:4

Run metrics:
Total lines of code: 18
Total issues by severity: Medium: 2  High: 1
```

2.1.3 Remediating the Code

To pass SAST validation, we fix the code by reading secrets from environment variables, utilizing database parameter placeholders, and using system arrays in subprocess to explicitly disable shell command injection paths.

COLAB CELL 3 **Create Secure File**

```
%%writefile secure_app.py
import os
import sqlite3
import subprocess

# 1. SECURE: Dynamic configuration read
DB_PASSWORD = os.getenv("DATABASE_PASSWORD")

def safe_login(username, password):
    # 2. SECURE: Parameterized query binding
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username = ? AND password = ?"
    cursor.execute(query, (username, password))
    return cursor.fetchone()

def safe_ping_host(user_ip):
    # 3. SECURE: Array arguments execute safely without passing input through a
raw shell system
    print(f"Pinging server...")
    subprocess.run(["ping", "-c", "1", user_ip], capture_output=True, text=True,
check=False)
```

EXPECTED OUTPUT – CELL 3

```
Writing secure_app.py
```

COLAB CELL 4 **Run Bandit Scan Verification**

```
!bandit secure_app.py
```

EXPECTED OUTPUT – CELL 4

```
Run started: 2026-10-09 14:45:00

Test results:
No issues identified.

Code scanned:
Total lines of code: 19
Total issues by severity: Medium: 0  High: 0
```

2.2 Dynamic Application Security Testing (DAST)

2.2.1 Theory and Core Concepts

While SAST checks the *text layout* of our app, DAST requires a **live, actively executing target network environment**. DAST tools interact exclusively over application layers (HTTP/APIs), passing intentional attack payloads directly into active processing pipelines to observe structural runtime responses.

> **Network Note for Students:** Running an API inside Google Colab hosts the service on 127.0.0.1 (localhost) *inside Google's cloud machine*. You cannot open 127.0.0.1:8000 directly on your personal computer because your browser is looking at your own laptop, not the cloud machine. To allow external browser testing, we launch **Localtunnel** in the background so it doesn't freeze the environment, generating a secure public proxy URL.

> **Localtunnel Security Landing Page:** When you click the generated URL, you will see a security splash warning page designed to prevent anti-abuse. To unlock it:

1. Copy the **Endpoint IP** printed directly inside your Colab output dashboard.
2. Paste it into the "IP Address" box on the web page and hit submit.
3. Append /docs to the end of your public link to interact with the OpenAPI playground.

2.2.2 Deploying and Exposing the Insecure Live App

COLAB CELL 5 **Create Live Vulnerable Web Service**

```
%%writefile vulnerable_api.py
from fastapi import FastAPI, Query
import sqlite3
import os

app = FastAPI()

conn = sqlite3.connect("users.db", check_same_thread=False)
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password
TEXT)")
cursor.execute("INSERT INTO users VALUES ('admin', 'SuperSecret123')")
conn.commit()

@app.get("/login")
def login(username: str = Query(...), password: str = Query(...)):
    query = f"SELECT * FROM users WHERE username = '{username}' AND password =
'{password}'"
    cursor.execute(query)
    user = cursor.fetchone()
    if user:
        return {"status": "success", "message": f"Welcome {user}"}
    return {"status": "fail", "message": "Invalid credentials"}

@app.get("/ping")
def ping(ip: str = Query(...)):
    stream = os.popen(f"ping -c 1 {ip}")
    output = stream.read()
    return {"status": "executed", "output": output}
```

EXPECTED OUTPUT – CELL 5

```
Writing vulnerable_api.py
```

COLAB CELL 6 **Background Server Boot & Public Expose (Fixed Background Execution)**

```
# 1. Install dependencies
!pip install fastapi uvicorn > /dev/null 2>&1
!npm install -g localtunnel > /dev/null 2>&1

import subprocess
import time

# 2. Host application on internal cloud port 8000
server = subprocess.Popen(["uvicorn", "vulnerable_api:app", "--host",
"127.0.0.1", "--port", "8000"])
time.sleep(3)
print("Web server successfully running internally on port 8000.")

# 3. Fetch Colab's public IP (This is the password for localtunnel verification
page)
print("\n--- LOCALTUNNEL VERIFICATION CODE ---")
print("Copy this IP code if the website asks for an 'Endpoint IP' / 'Tunnel
Host':")
!curl -s https://localtunnel.me
print("--------------------------------------\n")

# 4. Launch localtunnel in the BACKGROUND using "nohup" and "&"
# This prevents the cell from locking up and keeps the script executable!
get_ipython().system('nohup lt --port 8000 > tunnel.log 2>&1 &')
time.sleep(3)

# 5. Read the generated link from the log file
with open("tunnel.log", "r") as f:
    print(f.read())
```

EXPECTED OUTPUT – CELL 6

```
Web server successfully running internally on port 8000.

--- LOCALTUNNEL VERIFICATION CODE ---
Copy this IP code if the website asks for an 'Endpoint IP' / 'Tunnel Host':
35.255.179.171
--------------------------------------

nohup: appending output to 'nohup.out'
your url is: https://loca.lt
```

2.2.3 Executing the DAST Scan

We create a script cell that acts like an automated security scanner hitting the internal app process directly. We include the bypass-tunnel-reminder custom header to programmatically pass through the Localtunnel anti-abuse screen.

COLAB CELL 7 Run Dynamic Attacks

```
import requests

BASE_URL = "http://127.0.0.1:8000"
# Custom header required to programmatically skip the Localtunnel reminder
splash page
headers = {"bypass-tunnel-reminder": "true"}

print("--- STARTING DAST SCAN AGAINST RUNNING APP ---\n")

# Probe 1: Exploiting raw SQL logic evaluation via malicious parameter string
sql_payload = {"username": "admin' OR '1'='1", "password": "wrong_password"}
response_sql = requests.get(f"{BASE_URL}/login", params=sql_payload,
headers=headers).json()
print(f"[DAST Probe Login Response]: {response_sql}")
if response_sql.get("status") == "success":
    print("❌ DAST ALERT: Vulnerability Confirmed! SQL Injection bypass
achieved.\n")

# Probe 2: Appending a terminal delimiter command chain (;) to dump internal
target files
cmd_payload = {"ip": "127.0.0.1; cat /etc/hostname"}
response_cmd = requests.get(f"{BASE_URL}/ping", params=cmd_payload,
headers=headers).json()
print(f"[DAST Probe Ping Output]: {response_cmd.get('output')[:60]}...")
if response_cmd.get("output") != "":
    print("❌ DAST ALERT: Vulnerability Confirmed! Command Injection executed
system terminal files.")

server.terminate()
```

EXPECTED OUTPUT – CELL 7

```
--- STARTING DAST SCAN AGAINST RUNNING APP ---

[DAST Probe Login Response]: {'status': 'success', 'message': "Welcome ('admin',
'SuperSecret123')"}
❌ DAST ALERT: Vulnerability Confirmed! SQL Injection bypass achieved.

[DAST Probe Ping Output]: PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from ...
❌ DAST ALERT: Vulnerability Confirmed! Command Injection executed system
terminal files.
```

2.2.4 Deploying and Testing the Remediated Live App

COLAB CELL 8 **Create Secure Live Web Service**

```
%%writefile secure_api.py
from fastapi import FastAPI, Query
import sqlite3
import subprocess

app = FastAPI()

conn = sqlite3.connect("users_secure.db", check_same_thread=False)
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password
TEXT)")
cursor.execute("INSERT INTO users VALUES ('admin', 'SuperSecret123')")
conn.commit()

@app.get("/login")
def login(username: str = Query(...), password: str = Query(...)):
    query = "SELECT * FROM users WHERE username = ? AND password = ?"
    cursor.execute(query, (username, password))
    user = cursor.fetchone()
    if user:
        return {"status": "success", "message": f"Welcome {user}"}
    return {"status": "fail", "message": "Invalid credentials"}

@app.get("/ping")
def ping(ip: str = Query(...)):
    try:
        result = subprocess.run(["ping", "-c", "1", ip], capture_output=True,
text=True, timeout=5)
        return {"status": "executed", "output": result.stdout + result.stderr}
    except Exception as e:
        return {"status": "error", "message": str(e)}
```

EXPECTED OUTPUT – CELL 8

```
Writing secure_api.py
```

COLAB CELL 9 **Boot Secure Server Ecosystem (With Secure Background Tunnel)**

```
import subprocess
import time

# Host secure application on internal cloud port 8001
secure_server = subprocess.Popen(["uvicorn", "secure_api:app", "--host",
"127.0.0.1", "--port", "8001"])
time.sleep(3)
print("Secure web server successfully running internally on port 8001.")

# Launch dynamic background tunnel for port 8001
get_ipython().system('nohup lt --port 8001 > tunnel_secure.log 2>&1 &')
time.sleep(3)

with open("tunnel_secure.log", "r") as f:
    print(f.read())
```

EXPECTED OUTPUT – CELL 9

```
Secure web server successfully running internally on port 8001.
nohup: appending output to 'nohup.out'
your url is: https://localtunnel.me
```

COLAB CELL 10 Executing Active DAST Verification Run

```
import requests

BASE_URL = "http://127.0.0.1:8001"
headers = {"bypass-tunnel-reminder": "true"}

print("--- STARTING DAST RE-SCAN AGAINST SECURE APP ---\n")

# Throw exact same SQL Payload at secure endpoint
sql_payload = {"username": "admin' OR '1'='1", "password": "wrong_password"}
response_sql = requests.get(f"{BASE_URL}/login", params=sql_payload,
headers=headers).json()
print(f"[DAST Probe Login Response]: {response_sql}")
if response_sql.get("status") == "fail":
    print("✅ DAST VERDICT: Attack Defended! The application safely rejected the
SQL payload.\n")

# Throw exact same Command Payload at secure endpoint
cmd_payload = {"ip": "127.0.0.1; cat /etc/hostname"}
response_cmd = requests.get(f"{BASE_URL}/ping", params=cmd_payload,
headers=headers).json()
print(f"[DAST Probe Ping Output]: {response_cmd.get('output').strip()}")
if "invalid" in response_cmd.get("output", "").lower() or "not known" in
response_cmd.get("output", "").lower():
    print("✅ DAST VERDICT: Attack Defended! Input treated strictly as an
argument instead of executing shell operations.")

secure_server.terminate()
```

EXPECTED OUTPUT – CELL 10

```
--- STARTING DAST RE-SCAN AGAINST SECURE APP ---

[DAST Probe Login Response]: {'status': 'fail', 'message': 'Invalid
credentials'}
✅ DAST VERDICT: Attack Defended! The application safely rejected the SQL
payload.

[DAST Probe Ping Output]: ping: 127.0.0.1; cat /etc/hostname: Name or service
not known
✅ DAST VERDICT: Attack Defended! Input treated strictly as an argument instead
of executing shell operations.
```